Available online at www.sciencedirect.com
ScienceDirect
European Journal of Operational Research 176 (2007) 177–192
EUROPEAN JOURNAL OF OPERATIONAL RESEARCH
www.elsevier.com/locate/ejor

Discrete Optimization

A graph-based hyper-heuristic for educational timetabling problems

Edmund K. Burke [a], Barry McCollum [b], Amnon Meisels [c], Sanja Petrovic [a], Rong Qu [a,*]

[a] *Automated Scheduling Optimization and Planning Group, School of CSiT, University of Nottingham, Nottingham NG8 1BB, UK*
[b] *School of Computer Science, Queen's University Belfast, Belfast BT7 1NN, UK*
[c] *Department of Computer Science, Ben-Gurion University, Beer-Sheva 84 105, Israel*

Received 21 September 2004; accepted 18 August 2005
Available online 21 November 2005

Abstract

This paper presents an investigation of a simple generic hyper-heuristic approach upon a set of widely used constructive heuristics (graph coloring heuristics) in timetabling. Within the hyper-heuristic framework, a tabu search approach is employed to search for permutations of graph heuristics which are used for constructing timetables in exam and course timetabling problems. This underpins a multi-stage hyper-heuristic where the tabu search employs permutations upon a different number of graph heuristics in two stages. We study this graph-based hyper-heuristic approach within the context of exploring fundamental issues concerning the search space of the hyper-heuristic (the heuristic space) and the solution space. Such issues have not been addressed in other hyper-heuristic research. These approaches are tested on both exam and course benchmark timetabling problems and are compared with the fine-tuned bespoke state-of-the-art approaches. The results are within the range of the best results reported in the literature. The approach described here represents a significantly more generally applicable approach than the current state of the art in the literature. Future work will extend this hyper-heuristic framework by employing methodologies which are applicable on a wider range of timetabling and scheduling problems.
© 2005 Elsevier B.V. All rights reserved.

Keywords: Heuristics; Graph heuristics; Hyper-heuristics; Tabu search; Timetabling

* Corresponding author. Tel.: +44 115 8566503; fax: +44 115 8467591.
E-mail address: rxq@cs.nott.ac.uk (R. Qu).

0377-2217/$ - see front matter © 2005 Elsevier B.V. All rights reserved.
doi:10.1016/j.ejor.2005.08.012

1. Introduction

1.1. Timetabling problems

Timetabling has attracted the attention of the operational research and artificial intelligence research communities for more than 40 years. The general timetabling problem comes in many different guises including nurse rostering (e.g. Cheang et al., 2003; Burke et al., 2004b), sports timetabling (e.g. Easton et al., 2004), transportation timetabling (e.g. Kwan, 2004) and university timetabling (Schaerf, 1999; Burke and Petrovic, 2002; Petrovic and Burke, 2004), etc. Further examples of papers dealing with these kinds of problems can be found in Burke and Ross (1996), Burke and Carter (1998), Burke and Erben (2001), Burke and De Causmaecker (2003) and Burke and Trick (2005). Perhaps the most widely studied class of timetabling problem is that of educational timetabling. A wide variety of papers describing a broad spectrum of educational timetabling methodologies has appeared in the literature over the years. Overviews of the literature can be found in the following papers (Carter and Laporte, 1996, 1998; Bardadym, 1996; Burke et al., 1997; Schaerf, 1999; Burke and Petrovic, 2002; Petrovic and Burke, 2004).

In a general educational timetabling problem, a set of events (e.g. courses and exams, etc.) need to be assigned into a certain number of timeslots (time periods) subject to a set of constraints, which often makes the problem very hard to solve in real-world circumstances. Constraints can usually be divided into two types:

- *Hard constraints* have to be satisfied under any circumstances. For example, in exam timetabling, two exams with common students involved cannot be scheduled into the same timeslot. Timetables with no violations of hard constraints are called *feasible* solutions.
- *Soft constraints* need to be satisfied as much as possible. For example, in exam timetabling, exams taken by common students often need to be spread out over the timeslots so that students do not have to sit in two exams that are too close to each other. Due to the complexity of the real-world timetabling problem, the soft constraints may need to be relaxed since it is not usually possible to generate solutions without violating some of them. Soft constraints are usually used within the cost evaluation function to evaluate how good the solutions are.

1.2. Approaches and techniques in timetabling problems

The early days of research in educational timetabling investigated such approaches as graph heuristics (see de Werra, 1985; Burke et al., 2004c) and integer linear programming (see Carter, 1986). Some of the early techniques are either impractical or too simple to solve complex or large timetabling problems. Constraint based techniques have been employed over the years to address timetabling problems (e.g. Deris et al., 1997; Banks et al., 1998; Nonobe and Ibaraki, 1998). Recently, meta-heuristic search techniques (see Glover and Kochenberger, 2003) have been investigated and seem to have been very successful in solving a variety of timetabling problems. These include tabu search (e.g. Costa, 1994; Di Gaspero and Schaerf, 2001), simulated annealing (e.g. Dowsland, 1998; Abramson et al., 1999) and evolutionary algorithms (e.g. Burke et al., 1996, 1998; Burke and Newall, 1999; Erben, 2001; Lewis and Paechter, 2004; Côté et al., 2005). Other new approaches and methodologies for timetabling problems have also been studied as more problem solving experience is collected and new technologies provide breakthroughs. These include case-based reasoning (Leake, 1996) on educational timetabling (Burke et al., 2000, 2001, 2003c, in press, 2006) and on nurse rostering (Beddoe and Petrovic, in press), fuzzy methodology on exam timetabling (Asmuni et al., 2005), and hyper-heuristics on timetabling (Burke et al., 2003b; Gaw et al., 2005; Burke et al., 2005a, 2006; Qu and Burke, 2005).

The present work concerns educational timetabling including both exam and course timetabling problems (Carter and Laporte, 1996, 1998). The state-of-the-art approaches in educational timetabling usually employ specially tailored heuristic/

meta-heuristic approaches (e.g. Carter et al., 1996; Di Gaspero and Schaerf, 2001; Caramia et al., 2001; Burke and Newall, 2003; Merlot et al., 2003; Socha et al., 2002; Asmuni et al., 2005; Abdullah et al., 2004; Burke et al., 2004a). These approaches invest a significant amount of development effort in the production of "fine tuned" techniques that are specially built for the particular problems in hand. The objective of the present paper is to develop an approach which is more widely applicable and fundamentally more general than the approaches mentioned above. Our goal is not to "beat" them but to obtain comparable results by only employing general and simple techniques which can be applied to a wider range of scheduling problems.

1.3. Hyper-heuristics on scheduling and timetabling problems

The development of hyper-heuristics is motivated by the goal of aiming at an increased level of generality for automatically solving a range of problems (see Burke et al., 2003a). A hyper-heuristic can be seen as an algorithm (on a *higher level*) which "picks" appropriate heuristics (at a *lower level*) to be applied to the problems in hand. A hyper-heuristic is concerned with the exploration of a search space of heuristics instead of dealing directly with solutions to the problem. In hyper-heuristic research on timetabling and scheduling, different techniques have been investigated as the low level and high level search strategies in solving the problems. We can categorize the work in hyper-heuristics (in terms of the "low level" heuristics employed) into two groups: improvement techniques and constructive techniques.

1.3.1. Improvement techniques within hyper-heuristics

In a hyper-heuristic, *move* strategies are usually employed as the low level heuristics to search for solutions to timetabling and scheduling problems. Kendall et al. (2002) employed choice functions by which appropriate low level heuristics (moving strategies) can be chosen and combined adaptively to search the solutions. Good results on project presentation problems were presented compared with the solutions generated by a random hyper-heuristic. A distributed choice function method was proposed by Gaw et al. (2005) within a hyper-heuristic for timetabling and scheduling problems.

Burke et al. (2003b) employed a tabu search as the high level heuristic to search through a space of moving strategies for university course timetabling and nurse rostering problems. Good results on both of the problems tested indicated the generality and efficiency of the hyper-heuristic approach. This approach is adopted and extended in Burke et al. (2005b) with the aim of investigating the learning of low level heuristics that are suitable and effective for individual objectives in multiple-objective space allocation and course timetabling problems. Promising results are obtained compared with the state-of-the-art approaches. A genetic algorithm methodology was employed as the high level approach by Han and Kendall (2003) and was tested on simulated course scheduling and student project presentation problems. Dowsland et al. (in press) introduced the tabu search hyper-heuristic, which was investigated in Burke et al. (2003b), within a simulated annealing method to search a set of low level heuristics (both neighborhood structures and sampling policies within the solution space) to determine the shipper sizes in transportation problems.

Burke et al. (2006) employed a case-based reasoning methodology as a heuristic selector for solving course timetabling problems. Problem information is modeled with the corresponding *good* meta-heuristics and stored in the case-based reasoning system. The new problems are solved by using the suggested meta-heuristics which worked well on solving previous similar problems.

1.3.2. Constructive techniques within hyper-heuristics

There are only a few papers which employ constructive techniques as low level heuristics within hyper-heuristics for timetabling and scheduling problems. Terashima-Marin et al. (1999) investigated employing genetic algorithms to evolve the configurations of constraint satisfaction methods on constructing problem solutions. The non-direct representations were suggested for genetic

algorithms to solve large and complex exam timetabling problems.

Ross et al. (2003) investigated a genetic-based hyper-heuristic employing four basic constructive heuristics on one dimensional bin packing problems. Optimal or near optimal results have been found and potential research issues were also discussed.

Another technique was studied by Asmuni et al. (2005) who employed fuzzy rules to combine graph heuristics to construct exam timetables.

Burke et al. (2006) employed case-based reasoning as a heuristic selector for solving exam timetabling problems. Problem solving situations and the corresponding constructive heuristics were stored in the case-based reasoning system. Solutions for new problems were constructed by repeatedly using the constructive heuristics suggested by the system. Benchmark exam timetabling problems were tested and the results were within the range of those generated by using the state-of-the-art approaches.

The graph-based hyper-heuristic (GHH) described in this paper is a constructive approach that employs different graph heuristics during the process of constructing the solution according to the different problem solving situations that might occur at particular times. Section 2 presents the GHH approach upon graph heuristics and a random ordering method. Some fundamental issues concerning the search space and solution space are also addressed. The investigation and experimental study of GHH and of a multi-stage GHH that are developed for both exam and course timetabling problems are presented in Section 3. Our conclusions and potential extensions of this work are presented in the final section.

2. The graph-based hyper-heuristic (GHH) approach

2.1. Graph heuristics

Graph heuristics are widely studied methods which were developed during the early days of research on timetabling problems (e.g. Welsh and Powell, 1967; Brelaz, 1979). For an introduction to such approaches see Burke et al. (2004c). They are used in sequential (or constructive) solution methods to order the events that are not yet scheduled according to the difficulties of scheduling them into a feasible timeslot (without violating any hard constraints). The difficulties are represented by the degrees of the vertices in the graph, which model the timetabling problem by representing the events as vertices and conflicts by edges. We say two events in timetabling problems have a *conflict* if they involve the same students. The difficulty of an event is represented by the number of conflicts it has with the others. The objective is to construct a timetable by scheduling the most conflicting events one by one into feasible timeslots, satisfying as many of the soft constraints as possible.

Within our graph-based hyper-heuristic, we employ graph heuristics to schedule a number of events at each step during the construction of the solution. The work presented in this paper investigates the following five low level heuristics, with and without randomness (introduced by a random ordering method):

- Least saturation degree first (SD). Events are ordered (in an increasing order) in terms of the number of feasible timeslots available in the partial solution at that time. The priorities (degrees) of events to be ordered and scheduled are changed dynamically as the solution is constructed.
- Largest color degree first (CD). Events are ordered (in a decreasing order) in terms of the number of conflicts (events with common students involved) that they have with those already scheduled in the timetable. The degrees of the events not yet scheduled are changed according to the situations encountered at each step of the solution construction.
- Largest degree first (LD). Events are ordered decreasingly by the number of conflicts they have with other events. This heuristic aims to schedule first those events which have the most conflicts.
- Largest enrollment first (LE). Events are ordered (in a decreasing order) by the number of students enrolled. This heuristic schedules

first those events with the largest numbers of students.
- Largest weighted degree first (LWD). Events are ordered (in a decreasing order) by the number of conflicts, each of which is weighted by the number of students involved. Among events with the same degree, this heuristic gives higher priority to those with a larger number of students involved.
- Random ordering (RO). Events that are not yet scheduled are ordered randomly. This basic heuristic brings some randomness into the scheduling, which sometimes produces better results in timetabling problems as it increases the exploration of the search space.

2.2. The graph-based hyper-heuristic framework

In the hyper-heuristic framework that we present in this paper, tabu search is employed to search for the list of low level heuristics (permutations of graph heuristics and a random ordering method), which are used to construct the complete solutions for timetabling problems. Tabu search has been widely employed for solving complex scheduling and optimization problems (Glover and Kochenberger, 2003). The basic approach searches the search space of the solutions by iteratively moving to the best neighborhoods of the current solution, whilst keeping a record of recently visited solutions which it cannot re-visit again for a certain number of steps (known as the tabu tenure).

Fig. 1 presents the pseudo-code of the tabu search within the hyper-heuristic approach we have developed. The search space of the tabu search (as the high level heuristic) consists of all of the possible permutations of the low level heuristics described in Section 2.1. A move in tabu search is to pick a new heuristic list that is obtained by randomly changing two of the heuristics in the previous heuristic list. The newly visited heuristic lists are added into the tabu list (which has a length of 9. i.e. the tabu tenure is 9). The determination of this value is based on our experiments and upon the value recommended from the literature (Reeves, 1996). Of course, what this means is that when the tabu search selects a heuristic list, that list cannot be re-visited within nine steps of the search. The objective of the tabu search is to find the heuristic list that generates the best quality solution for the timetabling problem under consideration. The search process of tabu search within the hyper-heuristic approach stops after a pre-defined number of iterations (i in Fig. 1), which we adapt according to the problem size. We set it as five times the number of events to keep the computational time low. Of course it is possible to perform more iterations if computational time is not a key issue in the problem being solved.

Each heuristic list selected by a move of the tabu search is used to construct a complete solution, whose penalty is fed back to the tabu search as a guide to the search in the following stages. As each heuristic in the heuristic list is employed to schedule a number (e in Fig. 1) of events into the timetable (the reason for doing this is explained below), the length of the heuristic list (k in Fig. 1) is then set as (number of events)/e.

To reduce the size of the search space of the hyper-heuristic and thus reduce its computational time, we added three mechanisms:

(1) As the low level heuristics are graph heuristics which are used to construct the solutions, a list of them may not guarantee to generate a feasible solution (see more details in Section 2.3). To reduce the time spent on implementing heuristic lists that generate unfeasible solutions, we construct a 'failed list', which represents a *flat memory*, to store the parts of heuristic lists that generate unfeasible schedules during the solution construction (in the hyper-heuristic approach). The initial "failed list" is set as empty and is updated after each step of the tabu search to store any new heuristic lists that cannot generate feasible solutions. When a new heuristic list is selected by a move of tabu search in the hyper-heuristic, it will be checked (before it is applied to construct a solution) to see if it matches those stored in the 'failed list' that led to unfeasible schedules. For example, if a part of the heuristic list '$h1h2h3\ldots$' (where $h1$, $h2$ and h3 are low level heuristics) is stored in the 'failed list' because $h3$ cannot generate a feasible schedule at that step, all of the heuristic lists selected later such as '$h1h2h3h4\ldots$' or '$h1h2h3h5\ldots$'

```
initial heuristic list hl = {h1 h2 h3 … hk}
//Begin of Tabu Search
for i = 0 to i = (5 * the number of events) //number of iterations
 h  = change two heuristics in hl //a move in Tabu Search
  if h does not match a heuristic list in 'failed list'
       if h is not in the tabu list //h is not recently visited
       for j = 0 to j = k //h is used to construct a complete solution
            schedule the first 2 events in the event list ordered using hj
            if no feasible solution can be obtained
              store h into the 'failed list' //update "failed list"
            else if cost of solution c < the best cost cg obtained
              save the best solution, cg = c //keep the best solution
              add h into the tabu list
              remove the first item from the tabu list if its length > 9
        hl = h
  //end if
  Deepest descent on the complete solution obtained
//end of Tabu Search
output the best solution with cost of cg
```

Fig. 1. Pseudo-code of tabu search within the graph-based hyper-heuristic.

can be ignored before being applied to construct a solution. This mechanism cuts away a large section of the search space by ignoring the non-valid heuristic lists before applying them to solve the problems and reduces significantly the computational time of the hyper-heuristic.

(2) At each step of the solution construction employing a list of heuristics, a number of events (set to 2 in our experiments because our initial testing indicated that this was an appropriate value) are scheduled by the current heuristic in the list (rather than scheduling just one event with the heuristic). This is motivated by the observation by Burke et al. (2006) that, when scheduling one event by a heuristic at each step of the solution construction, the heuristics in the best heuristic list found by the hyper-heuristic stay the same after a certain number of steps of scheduling. That is, the best heuristic lists found tend to appear in the form of '$h2h2h2\ldots h2h1h1\ldots h1\ldots$'. Scheduling a num-

ber of events at each step reduces the size of the search space of the hyper-heuristic without losing much quality in the solutions generated.

(3) The initial heuristic list of the hyper-heuristic approach contains only the saturation degree heuristic, in order to have a good starting point in the hyper-heuristic. This is because the saturation degree heuristic orders the events not yet scheduled according to the number of available timeslots, which changes dynamically during the search. It is potentially (and experimentally tested to be) more effective (more often) than the other heuristics that order the events in static lists (Carter et al., 1996). It is expected that the density of the appearance of saturation degree in the best heuristic list will be higher than that of the other low level heuristics.

2.3. Fundamental issues within the hyper-heuristic approach

Hyper-heuristics have been attracting recent attention in timetabling research (see Burke and Petrovic, 2002; Burke et al., 2003b; Petrovic and Burke, 2004). However, some fundamental underlying issues have not been addressed in depth in the literature. Before presenting the analysis of our experiments, we would like to discuss some of these issues.

As mentioned before, hyper-heuristics operate at a *higher level* than most meta-heuristic approaches in the literature. They operate on heuristics rather than directly on the solutions by indirectly choosing a certain low level heuristic which then operates on the events (which either moves the events in the timetable if the low level heuristics are moving strategies, or assigns the events chosen to timeslots if the low level heuristics are constructive heuristics—which is the approach studied here). Thus, it is necessary to distinguish between the search space of the hyper-heuristic and the solution space of the problem.

Fig. 2 presents the relationship between the search space of hyper-heuristics and the solution space of the problem. Note that the search space of the *hyper-heuristic* (on the left side of Fig. 2) represents a space of heuristics and the solution space of the *problem* (on the right side of Fig. 2) represents a space of potential actual solutions (timetables). The heuristics in the search space of the hyper-heuristic correspond to certain solutions of the problem. Of course, the heuristics (which are constructive heuristics studied here) selected by a move in the high level searching method may not guarantee the construction of a feasible solution. This is because the moves in the hyper-heuristic search concern the change of heuristics and not the actual assignment of the events. For example, in Fig. 2 we can see that heuristic A moves to heuristic B, or heuristic A moves to heuristic C (within the search space of hyper-heuristic). The corresponding solutions (b or c in the solution space that are constructed by B or C) in the solution space are not guaranteed to be feasible. The search space of the hyper-heuristic (which consists of permutations of heuristics) is very large, with a large number of heuristics that generate unfeasible solutions in the solution space of the problem. In the figure, A and C generate feasible solutions, while B generates an infeasible solution.

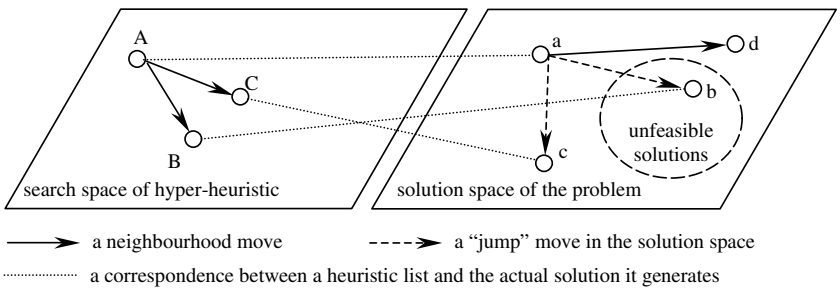


Fig. 2. The relation between the search space of the hyper-heuristic and solution space of the problem.

In Fig. 2 solutions *a* and *b* (which correspond to heuristics *A* and *B* that are in the same neighborhood in the search space of the heuristics), may not be in the same neighborhoods in the search space of solutions. This gives the hyper-heuristic the ability of *jumping* (not moving) within the problem solution space by moving among the neighborhoods defined at a higher level of search in the heuristic search space. In Fig. 2, neighborhood moves are represented as solid arrows and *jumping* moves are represented as dashed arrows. The correspondences between the heuristics and actual solutions are represented by dotted lines.

Also note that in Fig. 2, the solution space of the *problem* consists of all of the possible solutions that may be obtained by neighborhoods moves, while they may not correspond to any heuristic list in the search space of the hyper-heuristic. For example solution *d* may be within the neighborhood of solution *a* in the search space of solutions. However *d* may not have a corresponding heuristic in the search space of heuristics. Based upon the above observations, we propose a hypothesis here: the hyper-heuristic (which operates at a higher level of problem solving—solving the problems indirectly) may not *be able to* reach all of the solutions in the solution space of the problem.

A deepest descent local search method is employed within the hyper-heuristic after each move in the high level search. The deepest descent search tries to move the events to other timeslots in the timetable that is generated by a heuristic (searched for by the high level heuristic), aiming at improving the quality of the timetable as quickly as possible. This process terminates as soon as no events can be moved to improve the timetable. The high level heuristic will then make a move to another heuristic, which is used to construct another timetable. An illustrative example is presented in Fig. 3 in conjunction with the example shown in Fig. 2, where solution *d* in the search space of actual solutions might not corresponds to any heuristic in the search space of heuristic lists, while it might be in a neighborhood of solution *a* and thus may be visited by the deepest descent local search.

The deepest descent local search method is a simple but robust method, which does not introduce extra domain knowledge within the hyper-heuristic framework. The motivation for this is twofold: Firstly, the deepest descent in the search space of solutions tries to exploit the local areas (bringing the solutions obtained to their local optimum quickly), whilst the high level heuristic in the hyper-heuristic tries to explore the search space; secondly, the GHH thus is potentially able to explore more of the possible solutions for the problem.

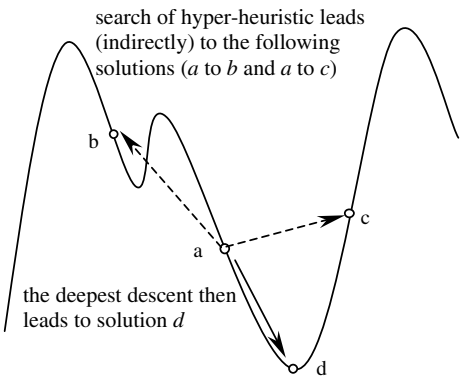


Fig. 3. An illustration of how the deepest descent local search could lead to solutions that are not represented by the heuristic lists (searched by the high level search).

3. GHH on exam and course timetabling problems

3.1. Real-world exam timetabling problems

We investigate our hyper-heuristic approach upon a set of graph heuristics by applying it to the benchmark exam timetabling problems that are presented in Carter et al. (1996). These are real-world problems that have been tested by many state-of-the-art approaches. The size of the problems ranges from 81 to 682 exams and from 611 to 18,419 students. The density of the conflict matrix, which gives the ratio of the number of conflicting exams over the overall number of exams, ranges from 0.06 to 0.42. The characteristics of the problems are presented in Table 1.

The hard constraints considered in these problems are represented by the "conflicts" of scheduling two exams with common students into the

Table 1
Characteristics of benchmark exam timetabling problems

	car91	car92	ear83	hec92	kfu93	lse91	sta83	tre92	ute92	uta93	york83
Exams	682	543	190	81	461	381	139	261	184	622	181
Students	16,925	18,419	1125	2823	5349	2726	611	4360	2750	21,266	941
Timeslots	35	32	24	18	20	18	13	23	10	35	21
Matrix density	0.13	0.14	0.27	0.42	0.6	0.6	0.14	0.18	0.8	0.13	0.29

same timeslot. The soft constraint is concerned with spreading out the students' exams over the timetable so that students will not have to sit exams that are too close to each other. The objective, which is the same as that presented in Carter et al. (1996), is to schedule all of the exams into the timeslots, while minimizing the cost on the violations of the soft constraint per student. The objective function of GHH which calculates the cost of violations $C(t)$ within solution t is presented in formula (1) below:

$$C(t) = \left(\sum_{s=0}^{4} w_s \times N_s \right) \Big/ S, \tag{1}$$

where $w_s = 2^s$, $s = 0, 1, 2, 3, 4$, is the weight that represents the importance of scheduling exams with common students either 4, 3, 2, 1, or 0 timeslots away in timetable t. N_s, $s = 0, 1, 2, 3, 4$, is the number of students involved in the violation of the soft constraint. S is the number of students in the problem.

The lower the cost, $C(t)$, the better the timetable is. We do not consider any infeasible solutions (i.e. those with violations of hard constraints).

3.1.1. A graph-based hyper-heuristic upon a different number of low level heuristics

We investigate here the effect of different low level heuristics on the behavior of the GHH for exam timetabling problems. This helps us to gain a deeper understanding about *general* hyper-heuristics which are applicable for a wider range of problems. The combinations of low level heuristics employed are based on the saturation degree heuristic with a different number of other heuristics with and without randomness. For all of the 11 problems presented in Carter et al. (1996), 3 runs with distinct seeds are carried out and the costs of the best solutions are presented in Table 2.

The values shown under the column "SL" and "SLR", "SCL" and "SCLR", and "SCLx" and "SCLxR" in Table 2 are the costs of solutions obtained by GHH upon two, three and all graph heuristics, with and without random ordering.

We assume that the larger the number of low level heuristics employed in the GHH, the larger the size of the search space (of the GHH) will be. This implies that it is possible to indirectly explore

Table 2
Costs of solutions obtained by GHH upon a different number of heuristics

Problem	SL	SLR	SCL	SCLR	SCLx	SCLxR	SCLxR(10*)
car91	5.78	5.67	5.52	**5.36**	5.65	5.43	5.39
car92	4.76	4.68	4.21	4.14	**4.53**	4.78	4.63
ear83	38.8	38.57	38.68	38.5	**37.92**	38.22	38.03
hec92	12.35	12.27	12.30	12.81	12.39	**12.25**	***12.11***
kfu93	16.21	15.79	15.37	15.23	15.67	**15.2**	***15.12***
Ise91	12.17	11.36	12.09	11.93	11.56	**11.33**	11.33
sta83	164.01	163.5	164.06	163.31	**158.19**	160.19	159.32
tre92	9.15	9.13	8.87	9.08	**8.75**	9.03	8.97
ute92	29.49	29.17	28.27	28.19	**28.01**	28.21	28.11
uta93	4.12	4.03	4.05	3.98	**3.88**	3.95	***3.78***
york83	44.54	42.67	42.44	42.37	**41.37**	42.01	41.52

S: saturation degree, L: largest degree, C: color degree; R: random ordering, Lx: largest weighted, largest enrollment and largest degree.

a larger part of the solution space of the problem (possibly containing more and better results). For example, the search space of GHH upon SCLxR should include the search space of GHH upon SCLR. This is apparent from Table 2, where the best results (in bold) are mostly obtained by GHH with a larger number of low level heuristics.

Although employing a larger number of low level heuristics in GHH tends to obtain a better performance, this is not always the case. For example, GHH upon "SCLx" outperforms "SCLxR", which employs a larger number of low level heuristics, on 10 out of 11 problems. The reason that GHH, when employing a smaller number of low level heuristics, sometimes outperforms the approach with a larger number of low level heuristics is precisely because the latter has a much larger search space. Some parts of the larger search space may not be explored within the same search time (i.e. number of iterations). To clarify this issue we carried out another set of experiments on "SCLxR" with a higher number of iterations (10 * number of events). The results obtained are presented in the last column entitled "SCLxR(10*)" in Table 2. We can observe that GHH upon "SCLxR" obtained better results (on 5 out of 11 problems than that of "SCLx") by being given more searching time. As the search space of GHH upon "SCLx" is a subset of that of GHH upon "SCLxR", it is possible to say that, given enough search time, the results obtained by GHH upon "SCLxR" will be at least not worse than that of GHH upon "SCLx".

We can also see that GHH upon "SL" and "SCL" with random ordering outperforms the same GHH without randomness, which is not the case for GHH upon "SCLx". When more searching time is given, GHH upon "SCLxR" performs much better, meaning that GHH employing heuristics with random ordering performs better than without randomness. We may then conclude, from the above observations, that the larger the number of low level heuristics in GHH, the better it may perform, provided a large enough amount of search time is given (but this is an important proviso).

3.1.2. Multi-stage GHH

The observations above provide the motivation for proposing a multi-stage GHH which employs a different number of heuristics in two stages. The multi-stage GHH employs "SCLR" in the first stage and "SCLxR" in the second stage to explore its search space as much as possible (based on the heuristic list obtained in the first stage). The reason for employing "SCLR" in the first stage of GHH is not only because it performs best, but also because it employs the four distinct low level heuristics (while the LE, LD and LWD are based on the same heuristic and may contribute similarly to LD).

The results obtained for the multi-stage GHH are presented in Table 3, together with the best results of the single-stage GHH in the last subsection. We also present the state-of-the-art approaches reported in the literature (Carter

Table 3
Results from the GHH, multi-stage GHH and the best results reported in literature on benchmark exam timetabling problems

	car91	car92	ear83	hec92	kfu93	lse91	sta83	tre92	ute92	uta93	york83
GHH (best)	5.36	4.53	37.92	12.25	15.2	11.33	158.19	8.92	28.01	2.88	41.37
Multi-stage GHH	5.4	4.74	38.84	13.11	15.99	12.43	159.9	9.02	30.65	3.04	43.51
Abdullah et al. (2004)	5.21	4.36	34.87	10.28	**13.46**	10.24	150.28	**8.13**	**24.21**	3.63	**36.11**
Asmuni et al. (2005)	5.20	4.52	37.02	11.78	15.81	12.09	160.42	8.67	27.78	3.57	40.66
Burke and Newall (2003)	**4.6**	**4.0**	37.05	11.54	13.9	10.82	168.73	8.35	25.83	**3.2**	36.8
Burke et al. (2004a)	4.8	4.2	35.4	10.8	13.7	10.4	159.1	8.3	25.7	3.4	36.7
Caramia et al. (2001)	6.6	6.0	**29.3**	**9.2**	13.8	**9.6**	158.2	9.4	24.4	3.5	36.2
Casey and Thompson (2003)	5.4	4.4	34.8	10.8	14.1	14.7	**134.7**	8.7	25.4	Inf	37.5
Carter et al. (1996)	7.1	6.2	36.4	10.8	14.0	10.5	161.5	9.6	25.8	3.5	41.7
Di Gaspero and Schaerf (2001)	6.2	5.2	45.7	12.4	18.0	15.5	160.8	10.0	29.0	4.2	41.0
Merlot et al. (2003)	5.1	4.3	35.1	10.6	13.5	10.5	157.3	8.4	25.1	3.5	37.4

et al., 1996; Di Gaspero and Schaerf, 2001; Caramia et al., 2001; Burke and Newall, 2003; Merlot et al., 2003; Asmuni et al., 2005; Abdullah et al., 2004). They are reported as the best results obtained by the corresponding approaches. We have not included an entry for the corresponding computational time because they are not reported in several of these papers.

We can observe that the multi-stage GHH does not outperform the single-stage hyper-heuristic presented in the last sub-section, although it obtains results that are not worse than that of GHH upon "SCLR". The reason may be that it starts from a smaller search space and this may limit the search towards a certain region of the search space.

3.1.3. Summarizing comments concerning GHH for exam timetabling problems

For all of the problems tested, GHH upon graph heuristics and random ordering obtained very good results that are within the range of the best results reported in the literature. By searching (on a higher level) the heuristic space, the GHH is capable of *jumping* (not moving) among the solution space of the problem by moving among the heuristic lists in its search space. We believe that this makes it a very efficient approach on difficult problems that have complex, especially disjointed, solution spaces.

Also note that except for Carter et al. (1996) where a number of different constructive methods with backtracking were employed, and (Asmuni et al., 2005) where constructing, un-scheduling and rescheduling are performed, all of the other approaches operate by improving on the initial complete solutions obtained beforehand. For the complex problems tested here, it is observed that, for some of the problem instances tested here, no feasible solutions can be obtained by the use of pure constructive graph heuristics (Carter et al., 1996). In contrast, the proposed hyper-heuristic method is constructive, starting from an empty solution. It is not dependent on initial solutions, which may sometimes affect the behavior of the other approaches. Most importantly, it is also effective for course timetabling problems (see below). Yet, the proposed method obtained competitive results with all state-of-the-art approaches for exam timetabling problems.

We have found that deepest descent local search after each move of the tabu search within the GHH approach often improves the solutions obtained and occasionally obtains a better final solution than that obtained by the tabu search alone. This may indicate that, during the problem solving, GHH sometimes reaches a point in the solution space (such as solution *a* in Figs. 2 and 3) and cannot go further to its local optimal (such as solution *d* in Figs. 2 and 3) unless moves upon the actual solutions are made. However, it is difficult to check this hypothesis thoroughly, due to the size of the search space for the solutions of complex timetabling problems. This is because the hypothesis concerns the coverage of the solution space by the search space of the hyper-heuristic (see Section 2.2).

3.2. University course timetabling problems

The proposed GHH method was also tested on eleven simulated benchmark course timetabling problems, proposed by the Metaheuristic Network.[1] This problem description is taken from Socha et al. (2002). The problems[2] need to assign 100–400 courses into a timetable of 5 days, 9 timeslots a day, while satisfying room features and capacity constraints. They are grouped into small, medium and large problems. The hard constraints that must be satisfied are:

1. no students can be scheduled to more than one event at a time;
2. the room meets all features required by the event;
3. room capacity is respected;
4. no more than one event is allowed per room and per timeslot.

Soft constraint violations are equally weighted and summed up within the cost function. They are presented below:

[1] http://www.metaheuristics.net/.
[2] http://iridia.ulb.ac.be/~msampels/ttmn.data/.

1. a student has a class in the last timeslot of the day;
2. a student has more than two classes in a row;
3. a student has only one class on a day.

3.2.1. GHH upon all low level heuristics

We employ exactly the same GHH approach with the same low level heuristics tested on the exam timetabling problems. The only changes made, in order to deal with these different problems, are the cost function which also evaluates the room constraints. The best results by 5 runs of GHH with distinct seeds upon all of the low level heuristics are presented in Table 4, which also presents the results of three approaches (Socha et al., 2002; Burke et al., 2003b) reported in the literature for comparison. For the "Tabu search hyper-heuristic" the best results out of 5 runs obtained are presented, for the "Local Search" and "Ant Algorithm" the average results out of 50 runs on small problems, 40 runs on medium problems and 10 runs on large problems are reported. We test our approach with the same number of runs as that of the "Tabu search hyper-heuristic" from Burke et al. (2003b) to make a more fair comparison. The term "x% Inf" in Table 4 indicates the percentage of runs which failed to obtain feasible solutions.

From Table 4 we can observe that our GHH approach obtains competitive results with the other three approaches on these course timetabling problems. For problem "Medium5", it obtained the best results among all approaches. It outperforms the "Local Search" method on all of the problems except "Medium1", "Medium2" and "Medium4". And for all of the problems tested it finds feasible solutions with all the five distinct seeds tested, which the "tabu search hyper-heuristic" and "local search" approaches failed to obtain.

3.2.2. Summarizing comments concerning GHH on course timetabling problems

The experimental results on GHH for course timetabling problems demonstrate its ability to work well on hard problems. We can observe that for problems "Medium5" and "Large", which are the hardest (highest costs obtained by the other approaches), GHH obtained the best and second-best results. The reason may be that the GHH is capable of *jumping* (while not *moving*) within the solution space by the moves within the search space of heuristic lists. This enables it to deal well with a broad range of hard problems (with a complex, and probably disjoint, solution space).

The deepest descent local search after each move of the tabu search improves the solution in most cases during the search of GHH. The best final results are sometimes from the improvement made by deepest descent local search on the solutions obtained by tabu search. This may strengthen our hypothesis about the coverage of

Table 4
Results of the GHH and the other three approaches in literature on benchmark course timetabling problems

	GHH upon six heuristics	Tabu search hyper-heuristic (Burke et al., 2003b)	Local search (Socha et al., 2002)	Ant algorithm (Socha et al., 2002)
Small1	6	**1**	8	**1**
Small2	7	**2**	11	3
Small3	3	**0**	8	1
Small4	3	**1**	7	**1**
Small5	4	**0**	5	**0**
Medium1	372	**146**	199	195
Medium2	419	**173**	202.5	184
Medium3	359	267	77.5% Inf	**248**
Medium4	348	169	177.5	**164.5**
Medium5	**171**	303	100% Inf	219.5
Large	1068	80% Inf 1166	100% Inf	**851.5**

the solution space vs. search space within our GHH approach.

For all the other approaches compared, initial solutions are required before the algorithms are performed. Our GHH solves the problem by starting from an empty solution in each move of tabu search. For the course timetabling problems tested here, the violations of soft constraints 2 (more than two consecutive courses) and 3 (single course assigned in one day) cannot be evaluated accurately until a complete solution is obtained. This is not true for the other approaches compared here, since complete solutions exist. In our GHH approach the evaluation can only be made approximately during the construction of the solution. This may limit the search of GHH when searching for globally optimal solutions, although the deepest descent local search method upon the complete solution after each step of tabu search can improve the timetables concerning these soft constraints.

4. Conclusions

The overall goal of this paper was to investigate a hyper-heuristic which operates at a higher level of generality than most of the current approaches studied in timetabling. Current state-of-the-art approaches are the result of a significant investment of effort in developing sophisticated and elaborate heuristics, which are "tailor made" for their particular problems. In our hyper-heuristic framework both the tabu search and graph heuristics are general methods that are widely applicable. We have presented a general constructive approach which is not dependent on the initial solution that the other approaches need to generate.

By employing simple and general heuristics in an intelligent way, the hyper-heuristic is capable of generating comparable results to those of special purpose approaches. The hyper-heuristic has the ability of selecting general heuristics, picking up the events that are most *difficult* to schedule (by different heuristics), in any given solution construction state. Although the goal of the present study is not to beat the specific approaches in the literature, the GHH works well on all of the problems and for one of the benchmark course timetabling problems, the hyper-heuristic we developed actually obtained the best result among all those reported in the literature. It is a simple, robust and very effective general approach that does not use any domain knowledge except in the cost function that deals with different constraints in different problems.

Experimental results indicate that the hyper-heuristic works more efficiently when using a larger number of low level heuristics. However, the size of the search space of GHH increases as well, which also increases the computational time. The multi-stage GHH was studied with the aim of getting good results in the first stage of GHH employing less low level heuristics and improving the results in the second stage employing more low level heuristics based on the heuristic lists obtained from the first stage. However the experimental results presented were worse than the single stage GHH due to the limitation of the starting points for the search of GHH.

Our hypothesis on the coverage of the solution space of the problem by searching within the search space of the hyper-heuristic is experimentally strengthened for both the exam and course timetabling problems. This hypothesis needs to be tested on a larger range of timetabling and scheduling problems.

5. Future work

Future work could use other simple heuristics or approaches, instead of tabu search, in exploring the search space of heuristics. A larger number of low level general heuristics can also be added to the hyper-heuristic framework to explore a larger section of the solution space of the problems. It might also be interesting to consider more than one low level heuristic when choosing the events to be scheduled at each step of the solution construction. Combining different low level heuristics at a single step of the solution construction may find more appropriate events to be scheduled. Due to the large search space of the hyper-heuristic, future work will also need to investigate the

impact of additional low level heuristics on computational time.

The hyper-heuristic approach described here represents a general and simple framework equipped with little domain knowledge, and may be easily applied to many other timetabling and scheduling problems with little effort. Future work should extend the same framework to other problems.

Acknowledgements

We would like to acknowledge EPSRC for sponsorship of this research (Grant No. GR/N36837/01). Professor Meisels's contribution was supported by an EPSRC Visiting Fellowship (GR/S53459/01). We would also like to thank the anonymous referees for their helpful comments.

References

Abdullah, S., Ahmadi, S., Burke, E.K., Dror, M., 2004. Investigating Ahuja-Orlin's large neighbourhood search for examination timetabling. Technical Report NOTTCS-TR-2004-8, School of CSiT, University of Nottingham, UK.

Abramson, D., Krishnamoorthy, M., Dang, H., 1999. Simulated annealing cooling schedules for the school timetabling problem. Asia-Pacific Journal of Operational Research 16, 1–22.

Asmuni, H., Burke, E.K., Garibaldi, J., 2005. Fuzzy multiple ordering criteria for examination timetabling. In: Burke, E.K., Trick, M. (Eds.), Selected Papers from the 5th International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 3616, pp. 344–354.

Banks, D., Beek, P., Meisles, A., 1998. A heuristic incremental modelling approach to course timetabling. In: Proceedings of the 12th Canadian Conference on Artificial Intelligence, pp. 16–29.

Bardadym, V., 1996. Computer-aided school and university timetabling: The new wave. In: Burke, E.K., Ross, P. (Eds.), Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 1153, pp. 22–45.

Beddoe, G., Petrovic, S., in press. Determining feature weights using a genetic algorithm in a case-based reasoning approach to personnel rostering. European Journal of Operational Research.

Brelaz, D., 1979. New methods to color the vertices of a graph. Communications of the ACM 22 (4), 251–256.

Burke, E.K., Carter, M. (Eds.), 1998. Selected Papers from the 2nd International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 1408.

Burke, E.K., De Causmaecker, P. (Eds.), 2003. Selected Papers from the 4th International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 2740.

Burke, E.K., Erben, W. (Eds.), 2001. Selected Papers from the 3rd International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 2079.

Burke, E.K., Newall, J., 1999. A multi-stage evolutionary algorithm for the timetabling problem. The IEEE Transactions on Evolutionary Computation 3 (1), 63–74.

Burke, E.K., Newall, J., 2003. Enhancing timetable solutions with local search methods. In: Burke, E.K., De Causmaecker, P. (Eds.), Selected Papers from the 4th International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2740, pp. 195–206.

Burke, E.K., Petrovic, S., 2002. Recent research directions in automated timetabling. European Journal of Operational Research 140 (2), 266–280.

Burke, E.K., Ross, P. (Eds.), 1996. Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 1153.

Burke, E.K., Trick, M. (Eds.), 2005. Proceedings of the 5th International Conference on the Practice and Theory of Automated Timetabling, Pittsburg PA, USA, August 2004. Lecture Notes in Computer Science, vol. 3616.

Burke, E.K., Newall, J., Weare, R., 1996. A memetic algorithm for university exam timetabling. In: Burke, E.K., Ross, P. (Eds.), Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 1153, pp. 241–250.

Burke, E.K., Jackson, S., Kingston, H., Weare, F., 1997. Automated timetabling: The state of the art. The Computer Journal 40 (9), 565–571.

Burke, E.K., Newall, J., Weare, R., 1998. Initialization strategies and diversity in evolutionary timetabling. Evolutionary Computation 6 (1), 81–103.

Burke, E.K., MacCarthy, B., Petrovic, S., Qu, R., 2000. Structured cases in case-based reasoning—Re-using and adapting cases for time-tabling problems. Knowledge-Based Systems 13 (2–3), 159–165.

Burke, E.K., MacCarthy, B., Petrovic, S., Qu, R., 2001. Case-based reasoning in course timetabling: An attribute graph approach. In: David, A., Watson, I. (Eds.), Case-Based Reasoning Research and Development, Lecture Notes in Artificial Intelligence, vol. 2080, pp. 90–104.

Burke, E.K., Kendall, G., Newall, J., Hart, E., Ross, P., Schulenburg, S., 2003a. Hyper-heuristics: An emerging

direction in modern search technology. In: Glover, F., Kochenberger, G. (Eds.), Handbook of Meta-Heuristics. Kluwer, pp. 457–474.

Burke, E.K., Kendall, G., Soubeiga, E., 2003b. A tabu search hyperheuristic for timetabling and rostering. Journal of Heuristics 9 (6), 451–470.

Burke, E.K., MacCarthy, B., Petrovic, S., Qu, R., 2003c. Knowledge discovery in hyper-heuristic using case-based reasoning on course timetabling. In: Burke, E.K., De Causmaecker, P. (Eds.), Selected Papers from the 4th International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2740, pp. 90–103.

Burke, E.K., Bykov, Y., Newall, J., Petrovic, S., 2004a. A time-predefined local search approach to exam timetabling problems. IIE Transactions on Operations Engineering 36 (6), 509–528.

Burke, E.K., De Causmaecker, P., Vanden Berghe, G., Van Landeghem, H., 2004b. The state of the art of nurse rostering. Journal of Scheduling 7 (6), 441–499.

Burke, E.K., Kingston, J., de Werra, D., 2004c. Applications to timetabling. In: Gross, J., Yellen, J. (Eds.), Handbook of Graph Theory. Chapman Hall/CRC Press, pp. 445–474.

Burke, E.K., Dror, M., Petrovic, S., Qu, R., 2005a. Hybrid graph heuristics within a hyper-heuristic approach to exam timetabling problems. In: Golden, B.L., Raghavan, S., Wasil, E.A. (Eds.), The Next Wave in Computing, Optimization, and Decision Technologies. Springer, pp. 79–91.

Burke, E.K., Landa Silva, J.D., Soubeiga, E., 2005b. Multi-objective hyper-heuristic approaches for space allocation and timetabling. In: Ibaraki, T., Nonobe, K., Yagiura, M. (Eds.), Meta-heuristics: Progress as Real Problem Solvers. Springer, pp. 129–158.

Burke, E.K., MacCarthy, B., Petrovic, S., Qu, R., in press. Multiple-retrieval case-based reasoning for course timetabling problems. Journal of Operations Research Society.

Burke, E.K., Petrovic, S., Qu, R., 2006. Case based heuristic selection for timetabling problems. Journal of Scheduling 2 (9).

Caramia, M., Dell'Olmo, P., Italiano, G., 2001. New algorithms for examination timetabling. In: Naher, S., Wagner, D. (Eds.), Algorithm Engineering 4th International Workshop, WAE 2000, Lecture Notes in Computer Science, vol. 1982, pp. 230–241.

Carter, M., 1986. A lagrangian relaxation approach to the classroom assignment problem. INFOR 27 (2), 230–246.

Carter, M., Laporte, G., 1996. Recent developments in practical exam timetabling. In: Burke, E.K., Ross, P. (Eds.), Selected Papers from the 1st International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 1153, pp. 3–21.

Carter, M., Laporte, G., 1998. Recent developments in practical course timetabling. In: Burke, E.K., Ross, P. (Eds.), Selected Papers from the 2nd International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 1408, pp. 3–19.

Carter, M., Laporte, G., Lee, S., 1996. Examination timetabling: Algorithmic strategies and applications. Journal of Operations Research Society 47, 373–383.

Casey, S., Thompson, J., 2003. GRASPing the examination scheduling problem. In: Burke, E.K., De Causmaecker, P. (Eds.), Selected Papers from the 4th International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2740, pp. 232–246.

Costa, D., 1994. A tabu search for computing an operational timetable. European Journal of Operational Research 76, 98–110.

Cheang, B., Li, H., Lim, A., Rodrigues, B., 2003. Nurse rostering problems: A bibliographic survey. European Journal of Operational Research 151 (3), 447–460.

Côté, P., Wong, T., Sabourin, R., 2005. Application of a hybrid multi-objective evolutionary algorithm to the uncapacitated exam proximity problem. In: Burke, E.K., Trick, M. (Eds.), Selected Papers from the 5th International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 3616, Springer, pp. 294–312.

de Werra, D., 1985. Graphs, hypergraphs and timetabling. Methods of Operations Research. 49, 201–213.

Deris, B., Omatu, S., Ohta, H., Samat, D., 1997. University timetabling by constraint-based reasoning: A case study. Journal of Operational Research Society 48 (12), 1178–1190.

Di Gaspero, L., Schaerf, A., 2001. Tabu search techniques for examination timetabling. In: Burke, E.K., Erben, W. (Eds.), Selected Papers from the 3rd International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2079, pp. 104–117.

Dowsland, K., 1998. Off the peg or made to measure. In: Burke, M., Carter, M. (Eds.), Selected Papers from the 2nd International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 1408, pp. 37–52.

Dowsland, K., Soubeiga, E., Burke, E.K., in press. A simulated annealing hyper-heuristic for determining shipper sizes. European Journal of Operational Research, doi:10.1016/j.ejor.2005.03.058.

Easton, K., Nemhauser, G., Trick, M., 2004. Sports scheduling. In: Leung, J. (Ed.), Handbook of Scheduling: Algorithms, Models, and Performance Analysis. CRC Press (Chapter 52).

Erben, W., 2001. A grouping genetic algorithm for graph coloring and exam timetabling. In: Burke, E.K., Erben, W. (Eds.), Selected Papers from the 3rd International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2079, pp. 132–158.

Gaw, A., Rattadilok, P., Kwan, R.S., 2005. Distributed choice function hyperheuristics for timetabling and scheduling. In: Burke, E.K., Trick, M. (Eds.), Selected Papers from the 5th International Conference on the Practice and Theory of Automated Timetabling. Lecture Notes in Computer Science, vol. 3616, pp. 51–70.

Glover, F., Kochenberger, G., 2003. Handbook of Metaheuristics. Kluwer.

Han, L., Kendall, G., 2003. Investigation of a tabu assisted hyper-heuristic genetic algorithm. In: Congress on Evolutionary Computation, Canberra, Australia, pp. 2230–2237.

Kendall, G., Cowling, P., Soubeiga, E., 2002. Choice function and random hyperheuristics. In: Proceedings of the 4th Asia-Pacific Conference on Simulated Evolution and Learning, pp. 667–671.

Kwan, R., 2004. Bus and train driver scheduling. In: Leung, J. (Ed.), Handbook of Scheduling: Algorithms, Models, and Performance Analysis. CRC Press (Chapter 51).

Leake, D. (Ed.), 1996. Case-based Reasoning: Experiences, Lessons and Future Directions. AAAI Press, Menlo Park, CA.

Lewis, R., Paechter, B., 2004. New crossover operators for timetabling with evolutionary algorithms. In: 5th International Conference on Recent Advances in Soft Computing, Nottingham, UK, vol. 5, pp. 189–195.

Merlot, L., Boland, N., Hughes, B., Stuckey, P., 2003. A hybrid algorithm for the examination timetabling problem. In: Burke, E.K., De Causmaecker, P. (Eds.), Selected Papers from the 4th International Conference on the Practice and Theory of Automated Timetabling, Lecture Notes in Computer Science, vol. 2740, pp. 207–231.

Nonobe, K., Ibaraki, T., 1998. A tabu search approach to the constraint satisfaction problem as a general problem solver. European Journal of Operational Research 106, 599–623.

Petrovic, S., Burke, E.K., 2004. University timetabling. In: Leung, J. (Ed.), Handbook of Scheduling: Algorithms, Models, and Performance Analysis. CRC Press (Chapter 45).

Qu, R., Burke, E.K., 2005. Hybrid variable neighborhood hyper-heuristics for exam timetabling problems. In: Metaheuristic International Conference 2005 (MIC2005), Vienna, August 2005.

Reeves, C.R., 1996. Modern heuristic techniques. In: Rayward-Smith, V.J., Osman, I.H., Reeves, C.R., Smith, G.D. (Eds.), Modern Heuristic Search Methods, pp. 1–25.

Ross, P., Marin Blasques, J.G., Schulenburg, S., Hart, E., 2003. Learning a procedure that can solve hard bin-packing problems: A new GA-based approach to hyper-heuristics. In: Cantú-Paz, E. et al. (Eds.), Genetic and Evolutionary Computation 2003, Lecture Notes in Computer Science, vol. 2724, pp. 1295–1306.

Schaerf, A., 1999. A survey of automated timetabling. Artificial Intelligence Review 13 (2), 87–127.

Socha, K., Knowles, J., Sampels, M., 2002. A max–min ant system for the university course timetabling problem. In: Proceedings of the 3rd International Workshop on Ant AlgorithmsLecture Notes in Computer Science, vol. 2463, pp. 1–13.

Terashima-Marin, H., Ross, P., Valenzuela-Rendon, M., 1999. Evolution of constraint satisfaction strategies in examination timetabling. In: Genetic Algorithms and Classifier Systems 1999, pp. 635–642.

Welsh, D.J.A., Powell, M.B., 1967. The upper bound for the chromatic number of a graph and its application to timetabling problems. The Computer Journal 11, 41–47.